

Advanced URL Shortener Web Application Report

Introduction

The Advanced URL Shortener Web Application is a full-stack web development project developed using Python and the Flask framework. The main purpose of this application is to convert long and difficult URLs into shorter and easy-to-share links. In today's digital world, users frequently share website links through emails, social media platforms, and messaging applications. However, lengthy URLs are often inconvenient to copy, paste, and share. A URL shortener solves this issue by generating compact URLs that are easier to manage and distribute.

This project was designed not only to shorten URLs but also to provide authentication functionality where users can create accounts, log in securely, and manage their own URL history. The application stores all shortened URLs in a database so users can access them later whenever required. The project provided practical exposure to frontend development, backend development, database management, authentication systems, and full-stack web application architecture using Flask.

Project Objective

The primary objective of this project is to build a functional web application that can shorten URLs efficiently while providing user authentication and database storage functionalities. The application allows users to create accounts using a username and password, log into the system, shorten URLs, save them in the database, and view previously shortened URLs through a history page.

Another important objective of this project is to understand the integration of multiple technologies such as Flask, SQLAlchemy, SQLite, HTML, CSS, Bootstrap, and Flask-Login. Through this project, the focus was on learning how frontend and backend systems interact with each other and how user data can be securely managed within a web application.

The project also aimed to improve problem-solving and debugging skills while working with real-world web application functionalities such as URL validation, user authentication, database relationships, routing, redirection, and session handling.

Problem Statement

Long URLs are often difficult to share because they are lengthy, unattractive, and inconvenient to remember. When users need to send links through social media, messages, or emails, copying and managing large URLs becomes time-consuming and inefficient. A URL shortener helps solve this issue by converting long URLs into compact and user-friendly links.

In addition to shortening URLs, users may also want to save and manage their previously shortened URLs. Therefore, this project extends the basic URL shortener functionality by adding user authentication and database integration. Each user can create an account, log in to the application, shorten URLs, and view only their own URL history. This ensures better organization and personalized access to stored URLs.

Technologies Used

Several technologies were used in the development of this project. Python was used as the primary programming language because of its simplicity and strong support for web development frameworks. Flask was used as the backend web framework to handle routing, server-side logic, and integration between the frontend and database.

SQLAlchemy ORM was used to interact with the SQLite database. Instead of writing raw SQL queries, SQLAlchemy allowed database operations to be performed using Python classes and objects, making the code more readable and manageable. SQLite was selected as the database because it is lightweight, simple to configure, and suitable for beginner-level projects.

Flask-Login was used to implement user authentication and session management. This library simplified the process of handling user login, logout, and route protection.

The frontend of the application was developed using HTML, CSS, and Bootstrap. HTML was used to create the structure of web pages, CSS was used for styling, and Bootstrap was used to make the application responsive and visually appealing. JavaScript was also used for implementing the “Copy URL” functionality.

System Architecture

The application follows a client-server architecture. The frontend is responsible for collecting user input and displaying results, while the backend processes requests, interacts with the database, and returns responses.

The frontend consists of multiple pages including Signup, Login, Home, and History pages. Users interact with the frontend by entering data into forms such as usernames, passwords, and URLs.

The backend developed using Flask handles all application logic including authentication, URL shortening, database storage, and redirection. Flask routes receive requests from the frontend, process the data, and return appropriate responses.

The SQLite database stores all user information and URL data. Two main tables are used in the project: the User table and the URLMap table. The User table stores account information, while

the URLMap table stores original URLs, shortened URLs, and the user associated with each URL.

Project Workflow

The workflow of the application begins when a new user visits the web application. If the user does not already have an account, they can create one using the Signup page by entering a username and password. The application validates the username to ensure that it is unique and satisfies the required length conditions.

After successful signup, the user is redirected to the Login page where they enter their credentials. Once authenticated successfully, the user gains access to the main URL shortener page.

On the Home page, the user enters a long URL into the input field and clicks the “Shorten URL” button. The backend validates the entered URL to ensure it is properly formatted. If the URL is valid, the application generates a random alphanumeric short code and stores both the original URL and short code in the database.

The shortened URL is then displayed on the screen. The user can copy this URL using the “Copy URL” button implemented using JavaScript.

The application also provides a History page where users can view all URLs they previously shortened. Clicking on a shortened URL automatically redirects the user to the original website.

Database Design

The project uses SQLite as the database management system. Two database tables were created using SQLAlchemy models.

The first table is the User table, which stores user-related information including user ID, username, and password. The username field is unique to ensure that duplicate usernames are not allowed.

The second table is the URLMap table, which stores URL-related information including original URLs, shortened codes, and the user ID associated with each URL. A relationship was established between the User table and URLMap table using foreign keys so that each user can maintain a separate URL history.

This database structure allows the application to efficiently store, retrieve, and manage user-specific URLs.

Authentication System

Authentication is one of the major features implemented in this project. Flask-Login was used to handle user authentication and session management. The authentication system ensures that only logged-in users can access the URL shortener functionality and History page.

The Signup functionality validates user input before creating a new account. If a username already exists or does not satisfy the required length constraints, appropriate error messages are displayed.

During login, the application verifies the entered username and password against the database records. If the credentials are correct, the user session is created and access is granted to protected routes.

The Logout functionality destroys the current user session and redirects the user back to the login page. The `@login_required` decorator was used to restrict unauthorized users from accessing protected pages.

URL Shortening Logic

The URL shortening functionality was implemented using Python's random and string libraries. Whenever a user enters a valid URL, the application generates a random six-character alphanumeric code consisting of uppercase letters, lowercase letters, and digits.

This generated code is appended to the host URL to create the shortened URL. For example, a URL such as:

`https://www.google.com`

may be converted into:

`http://127.0.0.1:5000/aB12Cd`

The shortened URL is stored in the database along with the original URL. When users access the shortened URL, Flask identifies the corresponding original URL from the database and redirects the user to the destination website.

URL Validation

URL validation was implemented using the validators library in Python. Before shortening any URL, the application checks whether the entered URL follows proper formatting rules.

If the entered URL is invalid, the application displays an error message and prevents incorrect data from being stored in the database. This validation step improves data integrity and ensures reliable application behavior.

Challenges Faced

Several challenges were encountered during the development process. One of the initial challenges was understanding how Flask routes interact with HTML templates and forms. Managing form data and passing dynamic content between frontend and backend required careful implementation.

Another major challenge was implementing user authentication using Flask-Login. Understanding session management, login protection, and user loading mechanisms required extensive learning and experimentation.

Database integration using SQLAlchemy also presented challenges, especially while establishing relationships between database tables and performing CRUD operations.

Handling URL redirection and generating unique short codes were additional technical challenges. These issues were resolved through debugging, documentation research, and continuous testing.

Overall, these challenges significantly improved problem-solving skills and practical understanding of Flask-based application development.

Testing and Results

The application was thoroughly tested to ensure that all functionalities worked correctly. Signup validation was tested by attempting duplicate usernames and invalid username lengths. Login functionality was tested using both valid and invalid credentials.

The URL shortening functionality was tested using multiple valid URLs such as Google, GitHub, and YouTube links. The generated shortened URLs successfully redirected users to the original websites.

Database storage was verified through the History page, which displayed all previously shortened URLs associated with the logged-in user.

The application performed successfully across all core functionalities including authentication, URL shortening, redirection, URL history, and logout operations.

Learning Outcomes

This project provided valuable practical experience in full-stack web application development. Through this project, concepts such as Flask routing, template rendering, form handling, database integration, ORM usage, session management, and user authentication were learned in detail.

The project also improved frontend development skills through the use of HTML, CSS, and Bootstrap. Working with SQLAlchemy ORM helped in understanding database modeling and relationships.

Most importantly, this project improved debugging, logical thinking, and software development skills while working on a real-world web application.

Future Enhancements

Several advanced features can be added to improve this application further. Password hashing can be implemented to enhance security. Analytics can be added to track the number of clicks on shortened URLs. QR code generation can also be integrated to allow users to share URLs more conveniently.

Additional features such as custom short URLs, email verification, password reset functionality, REST API support, dark mode, and cloud deployment can also be added in future versions of the project.

Conclusion

The Advanced URL Shortener Web Application was successfully developed using Flask, SQLAlchemy, SQLite, Flask-Login, HTML, CSS, and Bootstrap. The application successfully performs URL shortening, user authentication, URL validation, database storage, URL history management, and redirection functionalities.

This project provided hands-on experience in developing a complete full-stack web application and significantly improved understanding of backend development, database integration, authentication systems, and frontend-backend interaction.

Overall, the project was a valuable learning experience and strengthened practical skills in Flask-based web development and software engineering.